Week 4: Logistic and Softmax Regression*

Instructor: Dr. Federico Fusco

October 22, 2025 - version 2

1 Classification with "Linear" Regression: Logistic Regression

We want to use our insight on linear regression for binary classification, i.e., in the supervised learning setting where the label set $\mathbf{Y}$ consists of only two classes, let's say $\{0, 1\}$. We can use linear functions as a "score" for the likelihood of a certain x to be associated with the label 1. To this end, we introduce the sigmoid function (see also Figure 1):

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The sigmoid function maps in $[0, 1]$, meaning that we can interpret its output as probabilities. Moreover, its derivative has a very convenient form:

$$\sigma'(z) = -\frac{1}{(1 + e^{-z})^2} e^{-z} = \frac{1}{1 + e^{-z}} \left(1 - \frac{1}{1 + e^{-z}}\right) = \sigma(z)(1 - \sigma(z))$$

In this section, we predict the probabilities with the following rule:

$$h_\theta(x) = \sigma(\phi(x)^T \theta) = \frac{1}{1 + e^{-\phi(x)^T \theta}}$$

Note

This is called a logistic function, and ideally tells us how likely a given class is to realize!

Given the prediction made by the logistic predictor, it is immediate to design the corresponding prediction: if $h_\theta(x) \geq 1/2$, then the predicted label is 1, otherwise, it is 0. Now the question is about what is a good loss for probabilities. Given two probabilities p and q over a discrete support, we define the Kullback-Leibler divergence (KL in short) as

$$D_{\mathrm{KL}}(p \,\|\, q) = \sum_i p_i \log \frac{p_i}{q_i},$$

where p_i and q_i are the probabilities of observing the ith outcome, and we adopt the convention that $0 \times \log \infty = 0 = 0 \times \log 0$. We have the following properties:

*These notes are meant as a teaching support for the students of the "Machine Learning" course for the Master's Degree in Engineering in Computer Science and Artificial Intelligence that I am teaching in Fall 2025. They are likely full of typos and imprecisions, so please get in touch with me in class if something feels off. From time to time, I may update new versions of these notes, so make sure to download the latest version.

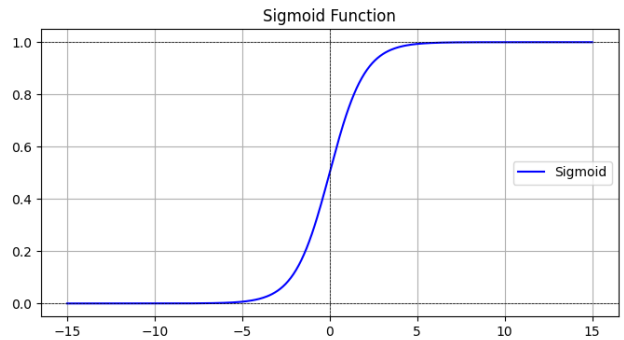

Figure 1: The Sigmoid Function σ

- The KL divergence is non-negative, and it is equal to zero only if $p = q$
- It does not depend on the actual values of the support of the distribution
- it weights the log of the probabilities by the probability of each class

This gives us a tentative objective: minimize the KL divergence between my predicted value $h_\theta(X)$ and the conditional expectation of Y given X. Let's now look at the empirical error over n samples of the form (x_i, y_i), where $y_i \in \{0, 1\}$. For point i, we are measuring the KL divergence between the predicted probability $h_\theta(x)$ (i.e., a Bernoulli with $h_\theta(x)$), and the (deterministic) distribution induced by the sample (which is either 0 or 1 with probability 1):

$$\begin{aligned}
&\sum_{i=1}^{n} y_i \log \frac{y_i}{h_\theta(x_i)} + (1 - y_i) \log \frac{1 - y_i}{1 - h_\theta(x_i)} \\
&\quad = \sum_{i=1}^{n} y_i \log y_i + (1 - y_i) \log(1 - y_i) - \sum_{i=1}^{n} y_i \log h_\theta(x_i) - (1 - y_i) \log(1 - h_\theta(x_i))
\end{aligned}$$

The red terms do not depend on the predictor, so our goal is to find θ that minimizes the error, that is also called *cross-entropy*:

$$-\sum_{i=1}^{n} y_i \log h_\theta(x_i) + (1 - y_i) \log(1 - h_\theta(x_i))$$

Alternatively, we could aim at maximizing the rescaled opposite of the cross entropy:

$$\hat{\ell}(\theta) = \frac{1}{n} \sum_{i=1}^{n} y_i \log h_\theta(x_i) + (1 - y_i) \log(1 - h_\theta(x_i)).$$

The function $\hat{\ell}$ is called the log-likelihood (often without the normalizing factor $1/n$). Unfortunately, there is no closed formula for finding $\max_\theta \hat{\ell}(\theta)$, but we can always resort to (stochastic) gradient descent.

$$\begin{aligned}
\frac{\partial \hat{\ell}}{\partial \theta_j} &= \frac{1}{n} \sum_{i=1}^{n} [y_i(1 - \sigma(\phi(x_i)^T \theta)) - (1 - y_i)\sigma(\phi(x_i)^T \theta)]\phi_j(x_i) \qquad \text{(as } \sigma'(z) = \sigma(z)(1 - \sigma(z)) \\
&= \sum_{i=1}^{n} [y_i - \sigma(\phi(x_i)^T \theta)]\phi_j(x_i)
\end{aligned}$$

Let's now look at the second derivative,

$$\frac{\partial \hat{\ell}}{\partial \theta_j \partial \theta_k} = -\frac{1}{n} \sum_{i=1}^{n} \sigma(\phi(x_i)^T \theta))(1 - \sigma(\phi(x_i)^T \theta)))\phi_j(x_i)\phi_k(x_i)$$

In matrix form, this means that the Hessian is $H = -1/n \sum_{i=1}^{n} \sigma(\phi(x_i)^T \theta))(1-\sigma(\phi(x_i)^T \theta)))\phi(x_i)\phi(x_i)^T$, therefore the matrix is negative semi-definite, which implies that the empirical log-likelihood is concave. So, doing gradient ascent will converge to a global maximum.

In particular, the sequential formula for gradient ascent will be:

$$\theta_{t+1} = \theta_t + \frac{\gamma_t}{n} \left(\sum_{i=1}^{n} (y_i - h_\theta(x_i))\phi(x_i) \right)$$

Similarly, for stochastic gradient ascent, we can sample u.a.r. one of the n points, and update with $(y_i - h_\theta(x_i))\phi(x_i)$

2 Multiclass Classification: Softmax Regression

We want to generalize the logistic regression to a multiclass setting, where the goal is to predict the label $Y = \{1, 2, \ldots, K\}$. We compute a linear score for each class $s_k(x) = \phi(x)^T \theta^{(k)}$, then we sample according to the soft-max function

$$\hat{p}_k(x) = \frac{e^{s_k(x)}}{\sum_\ell e^{s_\ell(x)}}.$$

Note

We can interpret the $\hat{p}_k(x)$ as the *probability* of observing label $Y = k$ when $X = x$. The function that maps scores to probabilities is called *soft-max*.

Once we have the $\hat{p}_k(x)$, the most sensible strategy is to output the class with the largest estimated probability. The empirical log-likelihood to maximize in this multiclass setting thus becomes

$$\hat{\ell}(\Theta) = \frac{1}{n} \sum_{i=1}^{n} \sum_{k=1}^{K} y_i^{(k)} \log(\hat{p}_k(x_i))$$

Note, in this setting, we are optimizing over all the k d-dimensional vectors $\theta^{(k)}$ for $k = 1, \ldots, K$. We use Θ to denote these K vectors. We can compute the gradient with respect to the generic $\theta^{(k)}$, to get:

$$\nabla_{\theta^{(k)}} \hat{\ell}(\Theta) = \frac{1}{n} \sum_{i=1}^{n} (\hat{p}_k(x_i) - y_i)\phi(x_i)$$

So, the gradient ascent formula for the kth weight vector becomes

$$\theta_t^{(k)} = \theta_{t-1}^{(k)} + \gamma_t \nabla_{\theta^{(k)}} \hat{\ell}(\Theta)$$

We conclude our study of softmax regression by arguing that the loss function is indeed convex. Before doing so, we prove a preliminary lemma.

Lemma 2.1. *The function $f : \mathbb{R}^K \to \mathbb{R}$ defined as follows is concave:*

$$f(z_1, \dots, z_K) = -\log \sum_{k=1}^{K} e^{z_k}.$$

Proof. We denote with S the sum of the exponential terms, namely $S = \sum_k e^{z_k}$ and start by computing the partial derivatives:

$$\frac{\partial f(z)}{\partial z_i} = -\frac{e^{z_i}}{S}, \quad \frac{\partial^2 f(z)}{\partial z_i^2} = \frac{e^{z_i}(S - e^{z_i})}{S^2}, \quad \frac{\partial^2 f(z)}{\partial z_i \partial z_j} = \frac{e^{z_i + z_j}}{S^2}$$

We can write the Hessian matrix of f in a convenient way: Denote with p the d-dimensional vector such $\pi_k = {}^{e^{z_k}}/_S$, we have the following:

$$H = \pi\pi^T - \mathrm{diag}(\pi),$$

where $\mathrm{diag}(\pi)$ is the $d \times d$ diagonal matrix whose diagonal contains the elements of π. We conclude the proof by showing that H is positive semi-definite, which implies convexity. For any vector $v \in \mathbb{R}^d$, indeed, we have:

$$\begin{aligned} v^T H v &= v^T(\pi\pi^T - \mathrm{diag}(\pi))v \\ &= v^T \pi\pi^T v - v^T \mathrm{diag}(\pi) v \\ &= -\left[\sum_{k=1}^{T} \pi_k v_k^2 - \left(\sum_{k=1}^{K} \pi_k v_k\right)^2\right] \leq 0. \end{aligned}$$

To see why the last inequality holds, it is enough to argue that $\sum_{k=1}^{T} \pi_k v_k^2 - \left(\sum_{k=1}^{K} \pi_k v_k\right)^2$ is the formula for the variance of the random variable that attains value v_k with probability π_k, and we know that the variance is always non-negative. □

Theorem 2.2. *The function $\hat{\ell}$ is concave.*

Proof. A possible strategy to prove this result consists of looking at the Hessian of $\hat{\ell}$, which is a $(K \cdot n) \times (K \cdot n)$ matrix. However, such an approach would lead to fairly involved calculations. We instead exploit the fact that $\hat{\ell}$ is the composition of simpler functions, which we show to be concave.

$$\begin{aligned} \hat{\ell}(\Theta) &= \frac{1}{n} \sum_{i=1}^{n} \sum_{k=1}^{K} y_i^{(k)} \log(\hat{p}_k(x_i)) \\ &= \frac{1}{n} \sum_{i=1}^{n} \sum_{k=1}^{K} y_i^{(k)} \left(s_k(x_i) - \log\left(\sum_{\ell=1}^{K} e^{s_\ell(x_i)}\right)\right) \\ &= \frac{1}{n} \sum_{i=1}^{n} \sum_{k=1}^{K} y_i^{(k)} s_k(x_i) - \frac{1}{n} \sum_{i=1}^{n} \sum_{k=1}^{K} y_i^{(k)} \log\left(\sum_{\ell=1}^{K} e^{s_\ell(x_i)}\right) \\ &= \frac{1}{n} \sum_{i=1}^{n} \sum_{k=1}^{K} y_i^{(k)} \phi(x_i)^T \theta^{(k)} - \log\left(\sum_{\ell=1}^{K} e^{s_\ell(x_i)}\right) \qquad \text{(By definition of } s_k\text{, and as } \textstyle\sum_k y_i^{(k)} = 1) \end{aligned}$$

The first term of the formula above is linear in the parameters $\theta^{(k)}$, by definition of the score functions s_k, therefore it is both concave and convex. The second term is also concave, as it is the composition of the concave function studied in Theorem 2.1 and the score functions, which are linear. All in all, $\hat{\ell}$ is concave as it is the sum of two concave functions.

□

3 Pointers

These notes are mainly based on Chapter 2 of Ng [2022]. Part of the exposition is also based on the final part of Chapter 4 of Géron [2019]. There is also a Notebook based on Géron [2019] on Classroom (Notebook 2).

References

Aurélien Géron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media, Inc, second edition edition, 2019.

Andrew Ng. Cs229 lecture notes. Online course notes, 2022. Available at `https://cs229.stanford.edu/notes2022fall/main_notes.pdf`.